

Module 10 — Environnements R & Python

Acte IV — R & Python pour l'analyse de données

Niveau : Débutant · Durée estimée : 2h30

Objectifs du module

À la fin de ce module, tu sauras :

- Comprendre **pourquoi R et Python** sont les outils incontournables du Data Analyst
- Installer et configurer **Python** (via Anaconda) et **R** (via RStudio)
- Naviguer dans **Jupyter Notebook** et **RStudio**
- Écrire tes **premiers scripts** dans les deux langages
- Choisir le bon outil selon la situation

Mise en contexte — Abidjan, cabinet Bamba & Associés

Après avoir maîtrisé Excel et SQL, Kouassi Ama — notre analyste junior — reçoit une nouvelle mission :

"Ama, nos clients veulent des analyses plus poussées : tendances, prévisions, visualisations avancées. Excel ne suffit plus. Il te faut apprendre Python ou R." — M. Bamba, Directeur

Ama se pose la même question que toi : **par où commencer ?**

1. Pourquoi R et Python ?

1.1 Le problème des limites d'Excel

Excel est excellent pour :

- Les fichiers jusqu'à ~100 000 lignes
- Les tableaux croisés dynamiques
- Les graphiques rapides

Mais Excel **montre ses limites** quand :

SITUATION	LIMITE D'EXCEL	SOLUTION
Fichier de 500 000 lignes	Lent, plante	Python / R
Analyse répétée chaque mois	Tout refaire à la main	Script automatisé
Graphique personnalisé	Options limitées	matplotlib, ggplot2
Modèle de prévision	Formules complexes	scikit-learn, tidymodels
Partage du travail	Fichier .xlsx fragile	Notebook partagé

1.2 Python vs R — Le match

CRITÈRE	PYTHON 🐍	R 📊
Usage principal	Polyvalent (web, data, IA)	Statistiques & visualisation
Courbe d'apprentissage	Douce	Modérée
Bibliothèques data	pandas, numpy, seaborn	dplyr, ggplot2, tidyr
Visualisation	matplotlib, seaborn, plotly	ggplot2 (très puissant)
Machine Learning	scikit-learn (leader)	tidymodels, caret
Demande emploi Côte d'Ivoire	★★★★★	★★★
Communauté francophone	Grande	Présente (académique)

1.3 La bonne nouvelle

Tu n'as pas à choisir. Un bon Data Analyst connaît les deux.
Dans ce bootcamp : on apprend **les deux en parallèle**, côte à côte.

2. Installer Python avec Anaconda

2.1 Pourquoi Anaconda ?

Anaconda est une **distribution Python** qui installe d'un coup :

- Python
- Jupyter Notebook
- Les bibliothèques data essentielles (pandas, numpy, matplotlib...)
- Un gestionnaire d'environnements (conda)

👉 **Beaucoup plus simple** que d'installer Python seul puis les bibliothèques une par une.

2.2 Installation pas à pas

Étape 1 — Télécharger Anaconda

```
Site : anaconda.com/download  
Choisir : version Individual Edition (gratuite)  
OS : Windows / macOS / Linux
```

Étape 2 — Installer

```
Windows : double-cliquer sur le .exe → suivre l'assistant  
macOS   : double-cliquer sur le .pkg → suivre l'assistant  
Linux   : bash ~/Downloads/Anaconda3-*.sh
```

⚠️ **Important (Windows)** : cocher "Add Anaconda to PATH" pendant l'installation.

Étape 3 — Vérifier l'installation

Ouvrir un terminal (ou Anaconda Prompt sur Windows) et taper :

```
BASH  
python --version  
# Résultat attendu : Python 3.11.x  
  
conda --version  
# Résultat attendu : conda 23.x.x
```

2.3 Lancer Jupyter Notebook

```
BASH  
jupyter notebook
```

Cela ouvre automatiquement ton navigateur à l'adresse `http://localhost:8888`.

Interface Jupyter :



3. Installer R et RStudio

3.1 Deux installations distinctes

Contrairement à Python/Anaconda, R s'installe en **deux étapes** :

1. **R** — le langage (moteur)
2. **RStudio** — l'environnement de travail (interface)

Analogie : R = le moteur d'une voiture, RStudio = le tableau de bord.

3.2 Installation de R

```
Site      : cran.r-project.org
Cliquer  : "Download R for [votre OS]"
Choisir  : la version la plus récente (ex: R 4.4.x)
```

3.3 Installation de RStudio

```
Site      : posit.co/download/rstudio-desktop
Choisir  : RStudio Desktop – Open Source (gratuit)
```

3.4 Interface RStudio — Les 4 panneaux

 Script / Source (écrire le code)	 Environment (variables créées)
> Console (exécuter, tester)	 Plots / Files (graphiques, aide)

PANNEAU	RÔLE
Source	Écrire et sauvegarder ton script
Console	Tester du code ligne par ligne
Environment	Voir les variables et données chargées
Plots/Files	Visualiser les graphiques, naviguer les fichiers

3.5 Vérifier l'installation

Dans la **Console** RStudio, taper :

```
R
R.version.string
# [1] "R version 4.4.x (2024-...)"
```

4. Maîtriser Jupyter Notebook

4.1 Les deux types de cellules

TYPE	USAGE	RACCOURCI
Code	Écrire du Python exécutable	Y (en mode commande)
Markdown	Écrire du texte formaté	M (en mode commande)

4.2 Raccourcis essentiels

Mode commande (appuyer **Esc** d'abord) :

RACCOURCI	ACTION
A	Insérer une cellule au-dessus
B	Insérer une cellule en-dessous
D D	Supprimer la cellule
M	Convertir en Markdown
Y	Convertir en Code
Shift+Enter	Exécuter et passer à la suivante
Ctrl+Enter	Exécuter et rester sur la cellule

4.3 Structure d'un notebook bien organisé

```
mon_analyse.ipynb
|
├─ [Markdown] # Titre de l'analyse
├─ [Markdown] ## 1. Contexte
├─ [Code]      import pandas as pd
├─ [Code]      df = pd.read_csv("data.csv")
├─ [Markdown] ## 2. Nettoyage
├─ [Code]      df.dropna()
├─ [Markdown] ## 3. Analyse
└─ [Code]      df.describe()
```

💡 **Bonne pratique** : alterner cellules Markdown (explication) et Code (calcul). Ton notebook devient ainsi un **rapport vivant** lisible par n'importe qui.

5. Premiers pas en Python

5.1 Variables et types de base

```
PYTHON
# Texte (string)
nom = "Kouassi Ama"
ville = "Abidjan"

# Nombre entier (int)
age = 28
nb_employes = 35

# Nombre décimal (float)
salaire = 450000.0

# Booléen (bool)
est_cadre = True

# Afficher
print(f"Bonjour, je suis {nom}, j'ai {age} ans")
# → Bonjour, je suis Kouassi Ama, j'ai 28 ans
```


5.2 Listes et dictionnaires

PYTHON

```
# Liste - une collection ordonnée
departements = ["RH", "Finance", "Commercial", "IT"]
salaires = [350000, 450000, 380000, 520000]

print(departements[0])    # RH (index commence à 0)
print(len(departements)) # 4

# Dictionnaire - clé: valeur
employe = {
    "nom": "Kouamé Jean",
    "departement": "Finance",
    "salaire": 480000,
    "contrat": "CDI"
}

print(employe["nom"])      # Kouamé Jean
print(employe["salaire"])  # 480000
```

5.3 Boucles et conditions

PYTHON

```
# Condition
salaire = 450000

if salaire >= 500000:
    print("Cadre supérieur")
elif salaire >= 350000:
    print("Cadre intermédiaire")
else:
    print("Agent de maîtrise")
# → Cadre intermédiaire

# Boucle for
equipe = ["Ama", "Jean", "Fatou", "Mamadou"]
for membre in equipe:
    print(f"Bonjour {membre} !")
# → Bonjour Ama !
# → Bonjour Jean !
# → ...
```

5.4 Fonctions

PYTHON

```
# Définir une fonction
def calculer_prime(salaire, taux=0.10):
    """Calcule la prime annuelle."""
    return salaire * taux * 12

# Appeler la fonction
prime_ama = calculer_prime(450000)
print(f"Prime annuelle : {prime_ama:,.0f} FCFA")
# → Prime annuelle : 540,000 FCFA

prime_jean = calculer_prime(480000, taux=0.15)
print(f"Prime Jean : {prime_jean:,.0f} FCFA")
# → Prime Jean : 864,000 FCFA
```

6. Premiers pas en R

6.1 Variables et types de base

R

```
# En R, on utilise <- pour assigner (ou = fonctionne aussi)
nom    <- "Kouassi Ama"
age     <- 28
salaire <- 450000.0
est_cadre <- TRUE

# Afficher
cat("Bonjour", nom, "\n")
# → Bonjour Kouassi Ama

print(paste("Salaire :", salaire, "FCFA"))
# → [1] "Salaire : 450000 FCFA"
```

6.2 Vecteurs (l'équivalent des listes Python)

```
R
# Vecteur – collection d'éléments du MÊME type
departements <- c("RH", "Finance", "Commercial", "IT")
salaires      <- c(350000, 450000, 380000, 520000)

# Accès (index commence à 1 en R, pas 0 !)
departements[1]  # "RH"
salaires[3]      # 380000

# Calculs vectoriels automatiques
salaires * 1.10  # augmentation de 10% pour tous
# → 385000 495000 418000 572000

mean(salaires)   # moyenne : 425000
sum(salaires)    # total   : 1700000
```

⚠ **Piège classique** : en R, les index commencent à 1 (pas 0 comme en Python).

6.3 Conditions et boucles

```
R
# Condition
salaire <- 450000

if (salaire >= 500000) {
  print("Cadre supérieur")
} else if (salaire >= 350000) {
  print("Cadre intermédiaire")
} else {
  print("Agent de maîtrise")
}
# → [1] "Cadre intermédiaire"

# Boucle for
equipe <- c("Ama", "Jean", "Fatou", "Mamadou")
for (membre in equipe) {
  cat("Bonjour", membre, "!\n")
}
# → Bonjour Ama !
# → Bonjour Jean !
```

6.4 Fonctions

```
R
# Définir une fonction
calculer_prime <- function(salaire, taux = 0.10) {
  return(salaire * taux * 12)
}

# Appeler
prime_ama <- calculer_prime(450000)
cat("Prime annuelle :", format(prime_ama, big.mark=","), "FCFA\n")
# → Prime annuelle : 540,000 FCFA
```

7. Python vs R — Comparaison côte à côte

Même tâche, deux langages — voici comment se traduit chaque concept :

CONCEPT	PYTHON	R
Assigner une variable	<code>x = 10</code>	<code>x <- 10</code>
Commentaire	<code># commentaire</code>	<code># commentaire</code>
Afficher	<code>print(x)</code>	<code>print(x)</code> ou <code>cat(x)</code>
Liste / Vecteur	<code>[1, 2, 3]</code>	<code>c(1, 2, 3)</code>
Index du 1er élément	<code>liste[0]</code>	<code>vecteur[1]</code>
Longueur	<code>len(liste)</code>	<code>length(vecteur)</code>
Moyenne	<code>sum(liste)/len(liste)</code>	<code>mean(vecteur)</code>
Condition	<code>if x > 0:</code>	<code>if (x > 0) {</code>
Boucle	<code>for i in liste:</code>	<code>for (i in vecteur) {</code>
Fonction	<code>def ma_fn(x):</code>	<code>ma_fn <- function(x) {</code>
Importer	<code>import pandas as pd</code>	<code>library(dplyr)</code>

Pont Excel → Python/R

Tu connais déjà Excel et SQL. Voici comment tes acquis se traduisent :

CE QUE TU FAIS DANS EXCEL	EN PYTHON	EN R
Ouvrir un fichier .xlsx	<code>pd.read_excel("data.xlsx")</code>	<code>read_excel("data.xlsx")</code>
Ouvrir un CSV	<code>pd.read_csv("data.csv")</code>	<code>read.csv("data.csv")</code>
Voir les données	<code>df.head()</code>	<code>head(df)</code>
Résumé statistique	<code>df.describe()</code>	<code>summary(df)</code>
Filtrer des lignes	<code>df[df["col"] > 100]</code>	<code>filter(df, col > 100)</code>
Calculer une moyenne	<code>df["col"].mean()</code>	<code>mean(df\$col)</code>
Créer un graphique	<code>df.plot()</code>	<code>plot(df\$col)</code>

CE QUE TU FAIS EN SQL	EN PYTHON (PANDAS)	EN R (DPLYR)
<code>SELECT col FROM table</code>	<code>df[["col"]]</code>	<code>select(df, col)</code>
<code>WHERE col > 100</code>	<code>df[df["col"] > 100]</code>	<code>filter(df, col > 100)</code>
<code>ORDER BY col</code>	<code>df.sort_values("col")</code>	<code>arrange(df, col)</code>
<code>GROUP BY + COUNT</code>	<code>df.groupby("col").size()</code>	<code>count(df, col)</code>
<code>GROUP BY + SUM</code>	<code>df.groupby("col") ["val"].sum()</code>	<code>summarise(group_by(df, col), \$=sum(val))</code>

Mini-exercices

Exercice 1 — Python : profil employé

En Python, crée les variables suivantes pour un employé de Bamba & Associés :

- `nom` = "Traoré Seydou"

- `departement` = "Commercial"
- `salaire` = 420000
- `annees_experience` = 5

Puis affiche : *"Traoré Seydou (Commercial) — 420,000 FCFA — 5 ans d'expérience"*

👉 Voir la réponse.....

PYTHON

```
nom = "Traoré Seydou"
departement = "Commercial"
salaire = 420000
annees_experience = 5

print(f"{nom} ({departement}) — {salaire:,} FCFA — {annees_experience}
ans d'expérience")
# → Traoré Seydou (Commercial) — 420,000 FCFA — 5 ans d'expérience
```

Exercice 2 — R : analyse des salaires

En R, crée un vecteur `salaires` contenant : 350000, 420000, 480000, 390000, 550000
Calcule et affiche : la **moyenne**, le **minimum** et le **maximum**.

👉 Voir la réponse.....

R

```
salaires <- c(350000, 420000, 480000, 390000, 550000)

cat("Moyenne :", mean(salaires), "FCFA\n")
# → Moyenne : 438000 FCFA

cat("Minimum :", min(salaires), "FCFA\n")
# → Minimum : 350000 FCFA

cat("Maximum :", max(salaires), "FCFA\n")
# → Maximum : 550000 FCFA
```

Exercice 3 — Python : classification des salaires

Pour chaque salaire de la liste `[280000, 350000, 450000, 520000, 680000]`,
affiche la catégorie :

- $< 300\,000 \rightarrow$ "Bas"
- $300\,000 - 499\,999 \rightarrow$ "Moyen"
- $\geq 500\,000 \rightarrow$ "Élevé"

👉 Voir la réponse.....

PYTHON

```
salaires = [280000, 350000, 450000, 520000, 680000]
```

```
for s in salaires:
    if s < 300000:
        categorie = "Bas"
    elif s < 500000:
        categorie = "Moyen"
    else:
        categorie = "Élevé"
    print(f"{s:,} FCFA → {categorie}")
```

```
# → 280,000 FCFA → Bas
# → 350,000 FCFA → Moyen
# → 450,000 FCFA → Moyen
# → 520,000 FCFA → Élevé
# → 680,000 FCFA → Élevé
```

Cheat Sheet — Python vs R

Syntaxe de base

ACTION	PYTHON	R
Variable	<code>x = 5</code>	<code>x <- 5</code>
String	<code>"texte"</code>	<code>"texte"</code>
Liste/Vecteur	<code>[1, 2, 3]</code>	<code>c(1, 2, 3)</code>
Booléen	<code>True / False</code>	<code>TRUE / FALSE</code>
Commentaire	<code># ...</code>	<code># ...</code>
Afficher	<code>print(x)</code>	<code>print(x)</code>
Longueur	<code>len(x)</code>	<code>length(x)</code>

Environnements

OUTIL	USAGE
Anaconda	Distribution Python tout-en-un
Jupyter Notebook	IDE Python orienté analyse/rapport
RStudio	IDE R avec 4 panneaux
Google Colab	Jupyter gratuit en ligne (pas d'installation)

Raccourcis Jupyter

RACCOURCI	ACTION
<code>Shift+Enter</code>	Exécuter la cellule
<code>A</code>	Cellule au-dessus
<code>B</code>	Cellule en-dessous
<code>D D</code>	Supprimer la cellule
<code>M</code>	Mode Markdown
<code>Y</code>	Mode Code



Cheat Sheets à télécharger

LANGAGE	RESSOURCE	LIEN
Python	Real Python Cheat Sheet	 Télécharger PDF
R	Base R Cheat Sheet (Harvard IQSS)	 Télécharger PDF



Imprime-les ou garde-les ouverts dans un onglet pendant que tu codes — tu t'y référeras constamment au début.



Quiz de validation

Question 1 — En R, comment accède-t-on au premier élément du vecteur `v <- c(10, 20, 30)` ?

- a) `v[0]`
- b) `v[1]`
- c) `v.first()`
- d) `first(v)`

👉 Voir la réponse.....

Réponse : b) `v[1]`

En R, les index commencent à 1 (contrairement à Python où ils commencent à 0). `v[1]` retourne `10`.

Question 2 — Quelle commande Python affiche les 5 premières lignes d'un DataFrame `df` ?

- a) `df.top(5)`
- b) `df.first(5)`
- c) `df.head()`
- d) `head(df, 5)`

👉 Voir la réponse.....

Réponse : c) `df.head()`

En Python/pandas, `df.head()` affiche les 5 premières lignes par défaut. En R, l'équivalent est `head(df)` — même nom, syntaxe différente.

Question 3 — Quelle est la différence entre R et RStudio ?

- a) RStudio est une version payante de R
- b) R est le langage (moteur), RStudio est l'interface de travail
- c) Ce sont deux langages différents
- d) RStudio remplace R, on n'a besoin que de RStudio

👉 Voir la réponse.....

Réponse : b)

R est le **langage** (moteur de calcul). RStudio est l'**environnement de développement** (IDE) qui rend R plus agréable à utiliser. Les deux sont gratuits et open source. Il faut installer R EN PREMIER, puis RStudio.

✅ **Résumé du module**

CE QUE TU AS APPRIS	DÉTAIL
Pourquoi Python & R	Limites d'Excel, complémentarité des deux outils
Installer Python	Anaconda → Jupyter Notebook
Installer R	CRAN → RStudio (4 panneaux)
Jupyter Notebook	Cellules Code/Markdown, raccourcis clavier
Bases Python	Variables, listes, dicts, boucles, fonctions
Bases R	Variables, vecteurs, boucles, fonctions
Pont Excel→Python/R	Traduction des actions Excel courantes
Pont SQL→Python/R	SELECT, WHERE, GROUP BY en pandas et dplyr

🚀 **Dans le module suivant**

Module 11 — Statistiques descriptives avec R & Python

On charge de vraies données ivoiriennes, on calcule les indicateurs clés (moyenne, médiane, écart-type, quartiles) et on commence à explorer avec **pandas** et **dplyr**.

